

Aurum Market: búsqueda semántica y control de catálogo

Construiréis un sistema capaz de recuperar productos por intención, aplicar restricciones de negocio y detectar altas potencialmente duplicadas. El resultado debe sostenerse con datos, métricas y una base vectorial operativa; no con un modelo generativo.

LA PREGUNTA QUE GUÍA EL TRABAJO

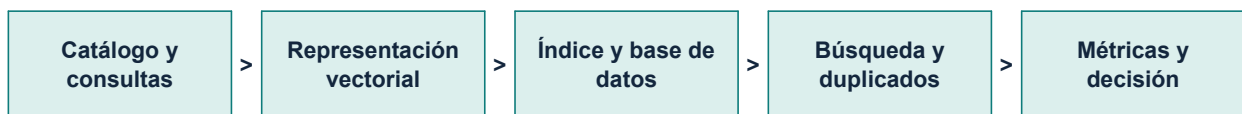
¿Qué decisiones permiten convertir un conjunto de embeddings en un servicio de descubrimiento fiable, mantenible y medible?

El reto conecta las tres capas trabajadas en la asignatura: la representación del significado, el índice que hace viable la búsqueda a escala y la base de datos que mantiene vectores, metadatos y cambios a lo largo del tiempo.

1. El caso de negocio

Aurum Market opera un catálogo con miles de referencias aportadas por vendedores distintos. La búsqueda actual depende demasiado de la coincidencia literal: una persona puede describir lo que necesita sin usar las palabras del título y terminar ante resultados irrelevantes. Al mismo tiempo, nuevas fichas llegan con títulos reordenados, marcas omitidas o descripciones ligeramente distintas, de modo que algunos productos se publican varias veces.

La empresa quiere una primera versión del motor de descubrimiento que pueda ejecutarse con recursos razonables, pero que no sea un prototipo imposible de mantener. Debe buscar, filtrar, actualizarse y dejar evidencia suficiente para explicar por qué se ha elegido una configuración concreta.



La misión

Diseñaréis e implementaréis un sistema que resuelva dos recorridos:

- **Descubrimiento semántico.** Dada una consulta, devolver un top-k ordenado y permitir que la búsqueda quede condicionada por metadatos como la marca.
- **Control de altas.** Dada una nueva ficha, recuperar el producto más parecido y decidir si existe un duplicado que deba revisarse antes de publicar.

FUERA DE ALCANCE

No se construirá un RAG, no se generarán respuestas y no se utilizará un LLM para resolver, etiquetar o reordenar consultas en tiempo de ejecución. Los modelos de embeddings sí forman parte del problema.

Qué se espera de una buena solución

No existe un proveedor obligatorio ni una configuración única. La calidad del trabajo dependerá de que las decisiones estén conectadas con el problema y contrastadas mediante experimentos. Un índice más sofisticado no compensa una representación deficiente; una métrica alta tampoco compensa una ingesta que duplica registros al repetirse.

2. Los datos

La entrega incluye un catálogo de 15.000 productos en español. Es lo bastante grande para que el índice y el almacenamiento importen, pero puede procesarse en un ordenador personal. También se incluye una muestra de 1.500 filas para levantar el recorrido completo antes de asumir el coste de la ingesta definitiva.

Fichero	Contenido	Uso esperado
catalogo_productos.csv.gz	15.000 productos con UUID estable, texto y metadatos.	Colección final sobre la que se realizarán las búsquedas.
catalogo_muestra.csv	1.500 registros con el mismo contrato.	Pruebas rápidas, conexión, esquema e ingesta.
consultas_desarrollo.csv	8 consultas con juicios de relevancia conocidos.	Comparar alternativas y fijar la configuración.
consultas_evaluacion.csv	12 consultas sin etiquetas visibles.	Entregar un top-10 reproducible.
consultas_filtradas.csv	4 consultas con una restricción de marca.	Demostrar búsqueda condicionada por metadatos.
eventos_catalogo.csv	24 altas, actualizaciones y eliminaciones ordenadas.	Probar idempotencia, mutaciones y visibilidad.
altas_desarrollo.csv	14 altas con etiqueta de duplicado.	Diseñar y calibrar la regla de decisión.
altas_evaluacion.csv	14 altas sin etiqueta visible.	Entregar la decisión y el candidato recuperado.

Relevancia graduada

Las consultas de desarrollo utilizan etiquetas ESCI. Para calcular nDCG se utilizará el mapeo Exact = 3, Substitute = 2, Complement = 1 e Irrelevant = 0. Recall@10 y MRR@10 deben declarar con claridad qué etiquetas se consideran relevantes. La elección puede discutirse, pero no puede cambiar silenciosamente entre experimentos.

Procedencia y trazabilidad

El catálogo deriva del Shopping Queries Dataset de Amazon Science, publicado bajo licencia Apache-2.0. El manifiesto conserva la procedencia, el contrato de IDs, los recuentos y los checksums. Las relevancias del conjunto ciego permanecerán reservadas hasta la corrección.

3. Construcción del sistema

3.1 Representación y baseline

Empezad por una referencia interpretable. El trabajo debe comparar el sistema denso con, al menos, un baseline léxico o exacto. Después elegid la representación textual, el modelo de embeddings, los prefijos que requiera y la normalización. No basta con citar la documentación del modelo: la elección debe apoyarse en los resultados de desarrollo y en las restricciones del caso.

- Documentad qué campos forman el texto que se codifica y cómo se sanean valores ausentes.
- Explicad la relación entre la métrica configurada, la normalización y el significado del score devuelto.
- Comparad al menos dos configuraciones relevantes. Cambiar solo el nombre del modelo sin analizar el resultado no constituye un experimento.

3.2 Índice y base de datos vectorial

La colección debe persistir, conservar metadatos y poder reconstruirse desde cero. Si el motor permite elegir el algoritmo ANN, la configuración debe hacerse explícita y relacionarse con lo aprendido sobre HNSW, IVF u otras familias. Si el proveedor oculta esa decisión, explicad qué control se pierde y cómo evaluaréis la fidelidad del ranking.

- Definid esquema, dimensión, métrica, IDs, metadatos y política de valores nulos.
- Implementad una ingesta por lotes e idempotente. Repetirla no debe aumentar el número de productos.
- Verificad el recuento final y el estado de indexación antes de aceptar consultas.
- Conservad la semántica del score nativo. No comparéis como si fueran equivalentes una distancia y una similitud.

3.3 Recuperación

El sistema expondrá una interfaz común -función, clase, API o comando- que reciba una consulta y devuelva resultados normalizados. Cada resultado incluirá como mínimo **product_id**, posición, título, metadatos necesarios y score.

- Búsqueda semántica global con **top_k** configurable.
- Búsqueda con filtro de marca ejecutado por la base de datos.
- Tratamiento explícito de colecciones vacías, filtros sin resultados y proveedores no disponibles.

4. Operaciones y control de catálogo

4.1 Cambios en la colección

Aplicad **eventos_catalogo.csv** respetando el orden de **sequence**. El proceso debe distinguir altas, actualizaciones y eliminaciones, y puede ejecutarse una segunda vez sin cambiar el estado final. Medid la visibilidad de, al menos, una operación de cada tipo mediante lecturas por ID y consultas vectoriales.

QUÉ DEMUESTRA ESTA PRUEBA

No se busca cronometrar qué proveedor es más rápido. Se busca verificar que una escritura confirmada acaba siendo observable por las rutas de lectura utilizadas y que el sistema sabe esperar, fallar o informar cuando no ocurre.

4.2 Altas potencialmente duplicadas

Utilizad **altas_desarrollo.csv** para diseñar una regla reproducible. Puede combinar score vectorial, margen respecto al segundo candidato, filtros de metadatos o una comprobación léxica, siempre que la base vectorial siga siendo el mecanismo de generación de candidatos.

- Justificad el umbral con los casos de desarrollo; no lo fijéis después de inspeccionar manualmente la evaluación.
- Una predicción positiva debe señalar el **product_id** concreto que se considera duplicado.
- Analizad por separado falsos positivos y falsos negativos: tienen costes de negocio diferentes.

4.3 Seguridad de las operaciones

Cualquier limpieza debe limitarse a recursos creados para la actividad y estar desactivada por defecto. No se incluirán credenciales en el repositorio. Si se utiliza un servicio cloud, la solución debe poder evaluarse sin que el corrector herede costes inesperados.

Elección del motor

Podéis utilizar Chroma, Weaviate, Milvus, Qdrant, Pinecone u otra base vectorial, siempre que la elección sea coherente y el núcleo de la solución utilice su SDK nativo. Una capa como LangChain puede aparecer como integración adicional, pero no sustituye la configuración, la observación de errores ni las operaciones administrativas del motor.

5. Evaluación y análisis

La evaluación debe servir para decidir, no para decorar el informe. Cada experimento conservará la configuración, las métricas y los IDs recuperados. Presentad una tabla comparativa compacta y utilizad los gráficos solo cuando hagan visible una relación que resulte difícil de leer en números.

Evidencia	Mínimo exigido	Interpretación
Calidad del ranking	nDCG@10, Recall@10 y MRR@10 sobre desarrollo.	Relacionar el cambio con modelo, texto, métrica o configuración.
Fidelidad ANN	Comparar IDs con un oráculo exacto sobre una muestra.	Separar pérdida del índice de error del modelo.
Latencia	p50 y p95 con entorno, calentamiento y número de repeticiones.	Describir esta ejecución; no construir un ranking entre nubes y local.
Filtros	Resultados que cumplan la marca en las cuatro consultas.	Verificar que el filtro forma parte de la consulta.
Duplicados	Precision, recall y F1 en desarrollo, con umbral documentado.	Explicar el coste de cada tipo de error.
Mutaciones	Recuento, lectura por ID, búsqueda y estado tras 24 eventos.	Demostrar idempotencia y visibilidad.

Atribución de errores

Elegid al menos tres fallos representativos y determinad dónde se originan. La explicación debe diferenciar, con evidencia:

- **Representación.** El vecino exacto ya es semánticamente malo.
- **Índice.** El oráculo exacto recupera un elemento que el ANN pierde.
- **Datos o filtros.** Falta información, el metadato es inconsistente o la consulta excluye el producto.
- **Persistencia o consistencia.** El estado leído no coincide aún con la escritura aplicada.

6. Entrega

La entrega debe poder ejecutarse en un entorno limpio siguiendo el README. Se valorará que el recorrido principal sea claro y corto: preparar entorno, levantar el motor, ingerir, evaluar y limpiar.

Repositorio

- Código o notebooks con una separación clara entre datos, embeddings, almacenamiento, búsqueda y evaluación.
- README con requisitos, instalación, variables, comandos, tiempos aproximados y solución de los fallos previsibles.
- Configuración reproducible y plantilla **.env** sin secretos.
- Pruebas mínimas de IDs, batching, filtros, mutaciones y formato de resultados.

Informe

Un PDF de hasta 10 páginas, sin contar anexos, que narre el problema, las alternativas probadas, la arquitectura final, los resultados y la decisión recomendada para Aurum Market. No debe convertirse en una transcripción del código: interesa entender qué se hizo, qué ocurrió y qué cambiaría al crecer el catálogo.

Artefactos de resultados

- **resultados_busqueda.csv** con 10 productos por consulta ciega.
- **resultados_duplicados.csv** con decisión, candidato y score.
- **metricas_desarrollo.json** con las métricas mínimas.
- Diagrama de arquitectura y fichero de configuración utilizado en la ejecución final.

Condiciones de comparabilidad

- No existe ventaja por utilizar cloud frente a local.
- No se exige interfaz web; una API, CLI o demostración en notebook bien construida es suficiente.
- No se puntúan funcionalidades ajenas al caso por el mero hecho de añadir complejidad.
- El catálogo completo es el recorrido evaluado; la muestra sirve para desarrollar y depurar.

7. Criterios generales de evaluación

La calificación combina funcionamiento y comprensión. Una solución puede ejecutar sin errores y, aun así, quedar incompleta si no mide la calidad o no sabe atribuir sus fallos. Del mismo modo, una discusión teórica no sustituye una colección operativa.

Bloque	Peso	Qué se observará
Problema y baseline	10 %	Contrato del sistema, baseline útil y decisiones conectadas con el caso.
Representación vectorial	20 %	Texto, embeddings, normalización, métrica y evidencia experimental.
Índice y base de datos	25 %	Configuración ANN, esquema, ingesta, persistencia, filtros y mutaciones.
Recuperación y evaluación	30 %	Ranking, duplicados, métricas, fidelidad ANN y atribución de errores.
Ingeniería y comunicación	15 %	Reproducibilidad, pruebas, seguridad operativa, arquitectura e informe.

Antes de entregar

- La ingesta completa puede repetirse sin aumentar el recuento.
- Las consultas filtradas nunca devuelven otra marca.
- Los eventos dejan exactamente el estado esperado.
- Los rankings ciegos contienen diez IDs únicos y válidos.
- La detección de duplicados señala un candidato cuando predice positivo.
- Las métricas pueden regenerarse desde un único comando.
- El repositorio no contiene claves, volúmenes ni datos reservados.

Fuentes del conjunto de datos

Amazon Science, *Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search*:
<https://github.com/amazon-science/esci-data>

Reddy et al. (2022), arXiv:2206.06588: <https://arxiv.org/abs/2206.06588>

CIERRE

El objetivo no es demostrar que una base vectorial devuelve algo. El objetivo es demostrar cuándo sus resultados son útiles, qué capa explica sus errores y qué decisiones permitirían llevar el sistema a producción con criterio.